

Code-Layout, Readability and Reusability

FinRelief AI — Empirical Development Phase Analysis

1. System Architecture & Module Boundaries

The FinRelief AI platform enforces a strict decouple-by-design architecture separating the Python FastAPI backend (``backend/``) from the React SPA frontend (``frontend/src/``). The client interacts exclusively via RESTful JSON APIs, eliminating HTML server-side rendering and maximizing component independence.

2. Backend Directory Layout (``backend/``)

The backend follows Python PEP 8 standards with modular domain separation:

- ``main.py``: Application initialization, middleware registration (CORS, SlowAPI rate limiting), and router assembly.
- ``database.py``: SQLAlchemy engine binding, session management, and DB connection pooling.
- ``models.py`` & ``schemas.py``: DB entity definitions (``User``, ``Loan``, ``StressSnapshot``, ``Letter``) and Pydantic validation schemas (``UserOut``, ``LoanOut``, ``SnapshotOut``, ``LetterOut``).
- ``auth.py``: JWT token issuance, password hashing (bcrypt), and authentication dependencies.
- ``routers/``: Isolated endpoint modules (``auth.py``, ``loans.py``, ``settlement.py``, ``letters.py``, ``dashboard.py``, ``demo.py``).
- ``utils/``: Self-contained business logic helpers (``calculations.py``, ``gemini_client.py``, ``pdf_generator.py``).

3. Frontend Directory Layout & UI Component Hierarchy (``frontend/src/``)

The frontend is organized into a domain-driven feature folder architecture:

- ``api/client.js``: Axios instance with request/response interceptors for JWT header attachment and global error handling.
- ``components/ui/``: Atomic, highly reusable UI primitives including `Button.jsx`, `Card.jsx`, `Modal.jsx`, `Input.jsx`, `Badge.jsx`, `MetricCard.jsx`, `Spinner.jsx`, and `EmptyState.jsx`.
- ``components/layout/``: Page framing elements (`AppShell.jsx`, `Sidebar.jsx`, `BottomNav.jsx`, `ProtectedRoute.jsx`).
- ``context/``: Global application state management (`AuthContext.jsx` and `ToastContext.jsx`).
- ``features/``: Page-level views scoped by functionality (`auth/`, `dashboard/`, `loans/`, `settlement/`, `letters/`).

4. Reusability & Clean Code Design Patterns

- **DRY UI Primitives**: Standardized design tokens using Tailwind CSS v4 wrapped inside reusable React components.
- **Stateless Math Utilities**: Pure, side-effect-free calculation functions in `calculations.py` and `stress.js` easily verifiable by unit tests.
- **Fallback Resiliency Pattern**: Gemini API integration in `gemini_client.py` provides automatic fallback to static legal templates on network or quota failures without throwing uncaught exceptions.